

Boid Szimuláció Gyorsítása OpenCL Segítségével

Losonczi Dominik

2026. május

1. Bevezetés

A dokumentum célja egy boid szimuláció bemutatása, valamint annak optimalizálása OpenCL segítségével. A boid algoritmus egy klasszikus mesterséges élet szimuláció, amely három alapvető szabályon alapul:

- Cohesion (összetartás)
- Alignment (irányigazítás)
- Separation (elkerülés)

2. Alap implementáció (CPU)

A CPU-alapú verzióban minden boid minden másik boidot vizsgál:

```
void update_boids() {
    for (int i = 0; i < NUM_BOIDS; i++) {
        for (int j = 0; j < NUM_BOIDS; j++) {
            if (i != j) {
                float dx = boids[j].x - boids[i].x;
                float dy = boids[j].y - boids[i].y;
                float dist = sqrt(dx*dx + dy*dy);

                if (dist < VIEW_RADIUS) {
                    // cohesion és alignment számítás
                }
            }
        }
    }
}
```

Ez a megoldás egyszerű, de nagy számú boid esetén lassú.

3. OpenCL alapú gyorsítás

A számítás GPU-ra került áthelyezésre OpenCL segítségével. A GPU párhuzamos feldolgozása lehetővé teszi, hogy minden boid külön szálon fusson.

3.1. Kernel példa

```
__kernel void update_boids_kernel(
__global const BoidCL* input,
__global BoidCL* output,
int numBoids
) {
int i = get_global_id(0);

if (i >= numBoids) return;

BoidCL self = input[i];

float avgX = 0.0f;
float avgY = 0.0f;
int count = 0;

for (int j = 0; j < numBoids; j++) {
    if (i == j) continue;

    float dx = input[j].x - self.x;
    float dy = input[j].y - self.y;
    float distSq = dx*dx + dy*dy;

    if (distSq < VIEW_RADIUS * VIEW_RADIUS) {
        avgX += input[j].x;
        avgY += input[j].y;
        count++;
    }
}

if (count > 0) {
    avgX /= count;
    avgY /= count;

    self.vx += (avgX - self.x) * 0.01f;
    self.vy += (avgY - self.y) * 0.01f;
}

self.x += self.vx;
self.y += self.vy;

output[i] = self;
```

}

4. Benchmark program

A teljesítmény mérésére külön benchmark program készült. Ennek célja, hogy a CPU és OpenCL verziót renderelés nélkül hasonlítsa össze.

A mérés során a program ugyanannyi szimulációs lépést futtat mindkét változattal:

```
for (int i = 0; i < TEST_FRAMES; i++) {  
    update_boids();  
}
```

OpenCL esetén:

```
for (int i = 0; i < TEST_FRAMES; i++) {  
    update_boids_opengl();  
}
```

A TEST_FRAMES értéke azt határozza meg, hogy hány szimulációs lépés fusson le egy mérés alatt. A benchmark eredménye egy CSV fájlba kerül.

5. Mérések

A mérésben 1000 és 10000 boid esetén lehet látni az eredményeket. A mérések tartalmazzák a módszert, a boid számát, a frame számot, a teljes futási időt, egy frame-re jutó futási időt és az fps-t.

5.1. Mérési eredmények

	A	B	C	D	E	F	G
1	run	method	num_boids	frames	total_time_sec	avg_frame_ms	fps
2	1	CPU	10000	1000	291.203	291.203	3.434031
3	1	OpenCL	10000	1000	2.163	2.163	462.3209
4	2	CPU	10000	1000	298.486	298.486	3.350241
5	2	OpenCL	10000	1000	2.204	2.204	453.7205
6	3	CPU	10000	1000	299.34	299.34	3.340683
7	3	OpenCL	10000	1000	2.188	2.188	457.0384
8	4	CPU	10000	1000	298.615	298.615	3.348794
9	4	OpenCL	10000	1000	2.18	2.18	458.7156
10	5	CPU	10000	1000	298.548	298.548	3.349545
11	5	OpenCL	10000	1000	2.188	2.188	457.0384

1. ábra. Mérési eredmények 10000 boid esetén

	A	B	C	D	E	F	G
1	run	method	num_boids	frames	total_time_sec	avg_frame_ms	fps
2		1 CPU	1000	1000	3.013	3.013	331.8951
3		1 OpenCL	1000	1000	0.255	0.255	3921.569
4		2 CPU	1000	1000	3.066	3.066	326.1579
5		2 OpenCL	1000	1000	0.273	0.273	3663.004
6		3 CPU	1000	1000	3.057	3.057	327.1181
7		3 OpenCL	1000	1000	0.262	0.262	3816.794
8		4 CPU	1000	1000	3.038	3.038	329.1639
9		4 OpenCL	1000	1000	0.257	0.257	3891.051
10		5 CPU	1000	1000	3.041	3.041	328.8392
11		5 OpenCL	1000	1000	0.255	0.255	3921.569

2. ábra. Mérési eredmények 1000 boid esetén

5.2. Elemzés

A mérések alapján az OpenCL verzió 10-12x gyorsabb számításra képes, mint a CPU változat, ennek ellenére a CPU sokkal stabilabban hozza az eredményeket, az OpenCL eredményeiben pedig megemelkedett a szórás.